adopted for managing the process was to embed it in the applications themselves, with the ability to activate it by calling a specific HTTP endpoint.

NOTE In production, you'll probably want a more automated and efficient way of refreshing configuration than explicitly triggering each application instance. When a remote Git repository backs your config server, you can configure a webhook that notifies the config server automatically whenever new changes are pushed to the repository. In turn, the config server can notify all client applications through a message broker like RabbitMQ, using Spring Cloud Bus. Chapter 14 will cover more scenarios for refreshing configuration in production.

ENABLING CONFIGURATION REFRESH

After committing and pushing the new configuration changes to the remote Git repository, you can send a `POST` request to a client application through a specific endpoint that will trigger a `RefreshScopeRefreshedEvent` inside the application context. You can rely on the Spring Boot Actuator project to expose the refresh endpoint by adding a new dependency in the build.gradle file for the Catalog Service project. Remember to refresh or reimport the Gradle dependencies after the new addition.

Listing 4.19 Adding dependency for Spring Boot Actuator in Catalog Service

```
dependencies {
  ...
  implementation 'org.springframework.boot:spring-boot-starter-actuator'
}
```

The Spring Boot Actuator library configures an `/actuator/refresh` endpoint that triggers a refresh event. By default, the endpoint is not exposed, so you have to enable it explicitly in the application.yml file of your Catalog Service project (catalog-service).